

Multi-Source Candidate Data Transformer — Technical Design

Aditya Grawal · adityapraveen109@gmail.com · Eightfold Engineering Intern (Jul–Dec 2026)

Pipeline

detect (by file extension, content-sniffed) → **parse** (one module per source type, never raises) → **normalize-at-ingest** (phone/date/skill/email/country) → **cluster & merge** (dedupe across sources, resolve conflicts, score confidence) → **project** (apply the runtime output config) → **validate** (check the projected output against the requested schema).

Sources covered

- **Structured:** Recruiter CSV export, ATS JSON blob (own field names, remapped via a small key-alias table).
- **Unstructured:** Resume (txt / PDF via pdfminer.six / DOCX via python-docx, regex + section-header heuristics), Recruiter notes (.txt, keyword-scan against a fixed skill vocabulary, lowest trust weight).
- GitHub/LinkedIn ingestion is descope'd for time but the *parse(path) → list[RawRecord]* interface is source-agnostic, so adding them later doesn't touch merge/project.

Canonical schema & normalization

Default schema follows the brief: `candidate_id`, `full_name`, `emails[]`, `phones[]` (E.164), `location` {city, region, country: ISO-3166 alpha-2}, `links`, `headline`, `years_experience`, `skills[]` {name, confidence, sources[]} (canonicalized via an alias table, e.g. "py"/"python3" → "Python"), `experience[]` {company, title, start, end: YYYY-MM}, `education[]`, `provenance[]` {field, source, method}, `overall_confidence`. All `normalize_*` functions are pure and never raise — bad input returns None rather than a crash or an invented value.

Merge & conflict-resolution policy

Matching: union-find clustering — two records are the same candidate if they share a normalized email OR phone OR an exact case-insensitive full name; email/phone matches are treated as strong keys and take precedence over name-only matches.

Conflict resolution: each source has a fixed trust weight (ATS JSON 1.0 > recruiter CSV 0.9 > resume 0.6 > recruiter notes 0.4 — structured/recruiter-entered data outranks heuristically-parsed prose). For a scalar field we keep the observation maximizing weight × confidence and log every other distinct value as provenance, so disagreement is visible, not silently dropped. List fields (emails, skills, experience, education) are unioned and deduped, never collapsed to one winner.

Confidence: a field's score is its winning weighted value, +0.1 (capped at 1.0) if ≥2 independent sources agree on the same normalized value. `overall_confidence` is the mean of populated field scores.

Runtime config / projection layer

`project.py` is the **only** module that reads the user-supplied config — ingestion and merge are fully config-agnostic, which is what makes "same engine, no code changes" actually true. The config can select a field subset, remap a field from a dotted/bracketed canonical path (e.g. `skills[]` . name), set per-field normalization (E.164, canonical skills), toggle confidence/provenance, and choose the on-missing policy: `null` (default) / `omit` / `error` (raises `ProjectionError` for a required-but-missing field). A separate `validate()` pass checks the projected output's shape/types against the requested field specs before it's returned.

Edge cases handled (and what's deliberately descope'd)

- Blank CSV rows and null-name ATS entries are dropped before clustering, never emitted as phantom profiles.
- Malformed JSON / unreadable files return [] from that source and the run continues end-to-end.
- Same person across 3+ sources with different field names and partially conflicting titles merges into one record, higher-trust source wins, loser values still recorded in provenance.
- **Descope'd:** recruiter_notes parser treats one .txt as one candidate's notes; a file mentioning two candidates can misattribute a phone number between them. A real fix needs per-mention text splitting, scoped out to keep the heuristic extractor simple and auditable under time pressure — documented in README rather than hidden.
- **Descope'd:** name-matching during clustering is exact (case-insensitive), not fuzzy — a deliberate precision-over-recall choice so two different people are never silently merged.